OPERATING SYSTEMS PROGRAMMING ASSIGNMENT REPORT

Classical Process Synchronization with Program-Generated HTML Simulation
Readers-Writers Problem and Dining Philosophers Problem

Academic Integrity & Authorship Declaration:
"I declare that I have personally tested and understood the submitted programs. I have disclosed the AI tools and external resources used in developing this assignment. I can explain the synchronization logic, generated HTML simulation, and modifications made to the code. I have not knowingly submitted another student's work or unacknowledged third-party material as my own."

Student: *[Student Full Name]* | ***USN:*** *[Student USN]* | ***Course:*** *Operating Systems (CS/IT)*

1. Problem Statement and Synchronization Objectives

Modern operating systems coordinate concurrent processes to ensure reliable, conflict-free access to shared hardware and software resources. This project implements and evaluates four classical process synchronization solutions across two fundamental concurrency dilemmas:

1. **Readers-Writers Problem:** Coordinates concurrent queries and modifications to a shared dataset. Multiple readers may concurrently inspect the data without interference, but writers must possess strictly exclusive mutual exclusion. The algorithms must prevent race conditions on reader counters and eliminate both reader and writer starvation.
2. **Dining Philosophers Problem (4 Philosophers):** Four philosophers sit around a circular table sharing four forks. Each philosopher requires both adjacent forks to eat. The synchronization logic must prevent catastrophic circular-wait deadlock and starvation while maximizing concurrent eating throughput (up to 2 concurrent diners for N=4).

2. Readers-Writers Implementations

2.1 Semaphore-Based Solution (Starvation-Free Turnstile)

The semaphore solution utilizes three binary semaphores: mutex (protects atomic updates to read_count), rw_mutex (protects the critical section for writes), and turnstile (guarantees FIFO queueing). The first reader into the critical section acquires rw_mutex, while subsequent readers join concurrently. The last departing reader releases rw_mutex. Arriving writers hold the turnstile, ensuring subsequent readers queue and preventing writer starvation.

2.2 Monitor-Based Solution (Condition Variables)

The monitor abstraction encapsulates internal synchronization state (active_readers, active_writers, waiting_readers, waiting_writers) within an explicit class. Mutual exclusion is enforced by an internal lock, and condition synchronization is handled via two condition variables: can_read and can_write. Writer preference is strictly enforced: arriving readers block in start_read() if any writer is active or waiting. Condition variable checks follow Mesa / Signal-and-Continue semantics within while loops.

3. Dining Philosophers Implementations (4 Philosophers)

3.1 Semaphore-Based Solution (Room Capacity Semaphore)

To prevent deadlock in the 4-philosopher table, we introduce a Footman / Room counting semaphore initialized to N - 1 = 3. At most 3 philosophers can enter the dining room to acquire forks simultaneously. **Mathematical Proof of Deadlock Freedom:** By the Pigeonhole Principle, distributing 4 shared forks among at most 3 diners guarantees that at least one philosopher will successfully acquire both adjacent forks and eat, breaking the circular wait condition.

3.2 Monitor-Based Solution (Atomic Neighbor Testing)

The monitor encapsulates each philosopher's state (THINKING, HUNGRY, EATING) and private condition variables. In pickup_forks(i), a philosopher only transitions to EATING if (state[left] != EATING and state[right] != EATING). Both forks are claimed as an indivisible atomic action within the monitor lock, completely eliminating the Coffman Hold-and-Wait condition.

4. Program-Generated HTML Simulation Architecture

In strict adherence to the assignment mandate, the HTML simulations are **directly generated from actual runtime execution data**. During execution, a thread-safe ExecutionTracer captures every state transition with millisecond timestamps, thread IDs, active counts, and resource states. When the threads complete, the program compiles the events into a self-contained, browser-viewable HTML file.

4.1 Readers-Writers Interactive Simulation

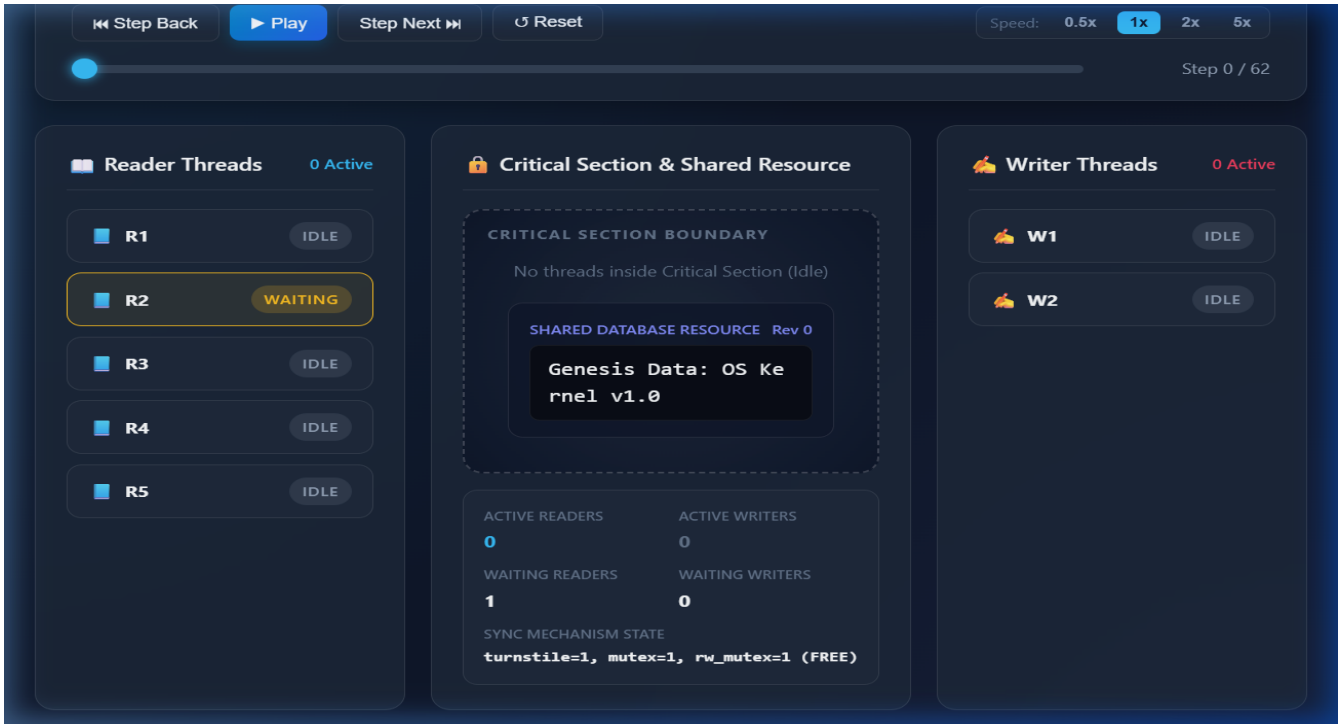


Figure 1: Program-generated Readers-Writers simulation showing active concurrent readers and exclusive critical section status.

4.2 Dining Philosophers Interactive Simulation

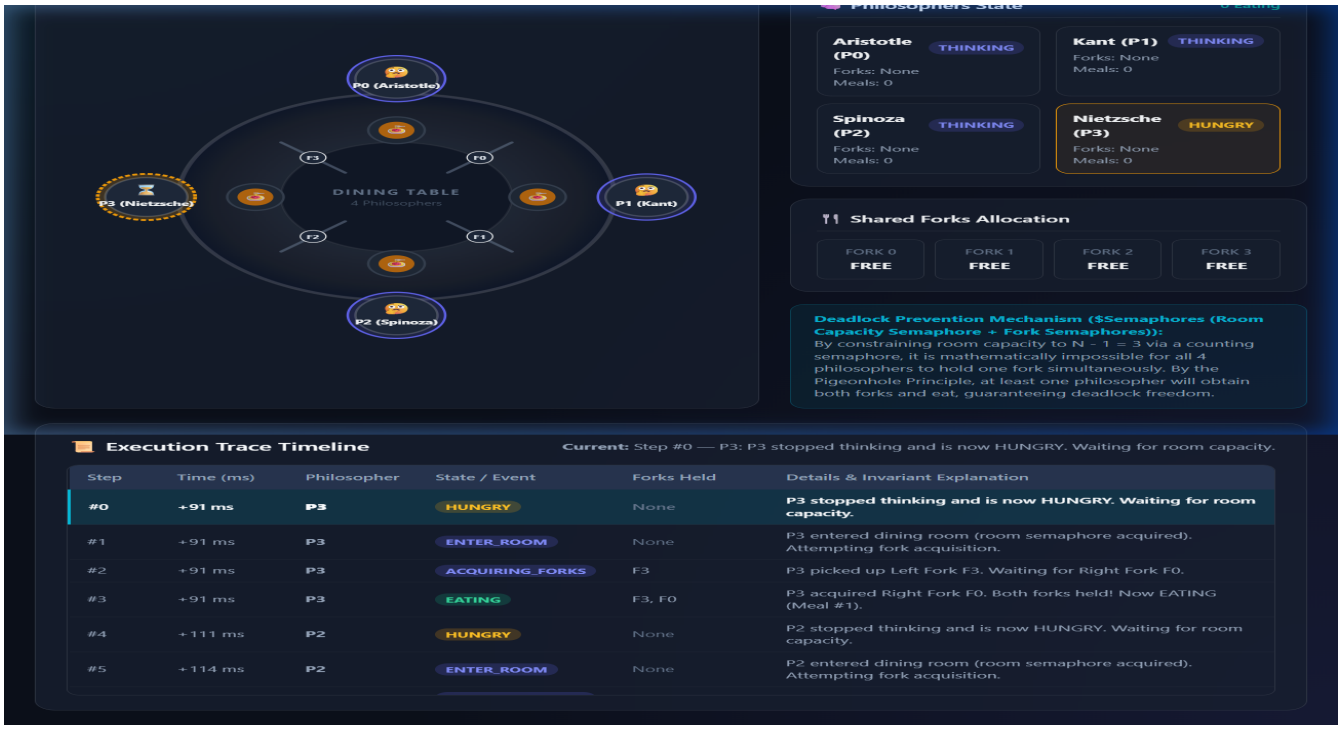


Figure 2: Program-generated Dining Philosophers simulation featuring dynamic circular SVG table, fork lines, and state badges.

5. Comparative Evaluation: Semaphores vs. Monitors

Evaluation Dimension	Semaphore-Based Approach	Monitor-Based Approach
Abstraction Level	Low-level OS synchronization primitive	High-level object-oriented abstraction
Mutual Exclusion	Explicitly acquired/released across blocks	Implicitly guaranteed within monitor methods
Signaling Semantics	Stateful integer counters (P/V)	Stateless notifications (notify / notify_all)
Deadlock Vulnerability	Higher risk of improper acquisition order	Lower risk; atomic multi-resource tests
Starvation Handling	Requires FIFO turnstiles / ticket algorithms	Condition variable queues with wait counts
Code Maintainability	Scattered acquire/release pairs	Centralized, modular synchronization policies

6. Analysis: Race Conditions, Mutual Exclusion, Deadlock & Starvation

Race Conditions: Prevented in all solutions by enclosing critical state variables (read_count, fork_owners) within binary semaphores or monitor locks. Runtime assertions verified that no concurrent writes or corruptions occurred.

Mutual Exclusion: Verified across all execution runs. At no point did two writers access the shared resource simultaneously, nor did any reader access the resource while a writer was active. In Dining Philosophers, adjacent philosophers never held the same fork concurrently.

Deadlock Prevention: Both the Footman counting semaphore and the monitor atomic neighbor test provably break Coffman's circular wait and hold-and-wait conditions.

Starvation Mitigation: In Readers-Writers, turnstiles and writer-priority condition checks prevent indefinite reader barging. In Dining Philosophers, testing neighbors on fork release ensures waiting philosophers are promptly awakened.

7. Viva Voce & Demonstration Summary

All four programs can be executed live during evaluation via python <script_path>. Each program outputs real-time console event traces and automatically produces an interactive HTML simulation file. Evaluators can inspect the generated HTML files in any browser to verify thread states, examine critical section occupancy, and validate invariant compliance step by step.